

Mock Code Challenge - National Parks (Object Relationships)

- Due No Due Date
- Points 1
- Submitting a website url

For this assignment, we'll be working with a national park planner-style domain.

We have three models: `NationalPark` , `Visitor` , and `Trip` .

For our purposes, a `NationalPark` has many `Trip` s, a `Visitor` has many `Trip` s, and a `Trip` belongs to a `Visitor` and to a `NationalPa`
`NationalPark` - `Visitor` is a many to many relationship.

Note: You should draw your domain on paper or on a whiteboard *before you start coding*. Remember to identify a single source of truth for your data.

Topics

- Classes and Instances
- Class and Instance Methods
- Variable Scope
- Object Relationships
- lists and list Methods

Instructions

To get started, run `pipenv install` while inside of this directory. Then run `pipenv shell` to jump into the shell.

Build out all of the methods listed in the deliverables. The methods are listed in a suggested order, but you can feel free to tackle the ones you th are easiest. Be careful: some of the later methods rely on earlier ones.

Remember! This code challenge has tests to help you check your work. You can run `pytest` to make sure your code is functional before submitting.

We've provided you with a tool that you can use to test your code. To use it, run `python debug.py` from the command line. This will start a `ipdb` session with your classes defined. You can test out the methods that you write here. You can add code to the `debug.py` file to define variables and create sample instances of your objects.

Writing error-free code is more important than completing all of the deliverables listed - prioritize writing methods that work over writing more methods that don't work. You should test your code in the console as you write.

Similarly, messy code that works is better than clean code that doesn't. First, prioritize getting things working. Then, if there is time at the end, refactor your code to adhere to best practices. When you encounter duplicated logic, extract it into a shared helper method.

Before you submit! Save and run your code to verify that it works as you expect. If you have any methods that are not working yet, feel free to leave comments describing your progress.

Deliverables

Write the following methods in the classes in the files provided. Feel free to build out any helper methods if needed.

Initializers and Properties

Visitor

- `Visitor __init__(self, name)`
 - Visitor is initialized with a name
- `Visitor property name`
 - Returns the visitor's name
 - Names must be of type `str`
 - Names must be between 1 and 15 characters, inclusive
 - Should **be able** to change after the visitor is instantiated

NationalPark

- `NationalPark __init__(self, name)`
 - `NationalPark` is initialized with a name, as a string
- `NationalPark` property `name`
 - Returns the national_park's name
 - Names must be of type `str`
 - Names length must be greater or equal to 3 characters
 - Should **not be able** to change after the national_park is instantiated
 - *hint: hasattr()*

Trip

- `Trip __init__(self, visitor, national_park, start_date, end_date)`
 - Trip is initialized with a `Visitor` instance, a `NationalPark` instance, a `start_date`, and an `end_date`
- `Trip` property `start_date`
 - Returns the trip's `start_date`
 - `Start_date` must be of type `str`
 - `Start_date` length must be greater or equal to 7 characters
 - Is in the format "September 1st"
 - Should **be able** to change after the trip is instantiated
- `Trip` property `end_date`
 - Returns the trip's `end_date`
 - `End_date` must be of type `str`
 - `End_date` length must be greater or equal to 7 characters
 - Is in the format "September 1st"
 - Should **be able** to change after the trip is instantiated

Object Relationship Methods and Properties

Trip

- `Trip` property `visitor`
 - Returns the Visitor object for that trip

- Must be of type `Visitor`
- `Trip` property `national_park`
 - Returns the `NationalPark` object for that trip
 - Must be of type `NationalPark`

Visitor

- `Visitor` `trips()`
 - Returns a list of all trips for that visitor
 - Trips must be of type `Trip`
- `Visitor` `national_parks()`
 - Returns a **unique** list of all parks that visitor has visited
 - Parks must be of type `NationalPark`

NationalPark

- `NationalPark` `trips()`
 - Returns a list of all trips at a particular national park
 - Trips must be of type `Trip`
- `NationalPark` `visitors()`
 - Returns a **unique** list of all visitors a particular national park has welcomed
 - Visitors must be of type `Visitor`

Aggregate and Association Methods

National Park

- `NationalPark` `total_visits()`
 - Returns the total number of times a park has been visited
 - Returns `0` if the park has no visits
- `NationalPark` `best_visitor()`
 - Returns the `Visitor` instance that has visited that park the most
 - Returns `None` if the park has no visitors

Visitor

- `Visitor total_visits_at_park(park)`
 - Receives a **NationalPark object** as argument
 - Returns the total number of times a visitor visited the park passed in as argument
 - Returns `0` if the visitor has never visited the park

Bonus: Aggregate and Association Method

- `NationalPark classmethod most_visited()`
 - Returns the `NationalPark` instance with the most visits.
 - Returns `None` if there are no visits.
 - *hint: will need a way to remember all `NationalPark` objects*
 - *hint: do you have a method to get the total visits for a particular `NationalPark` object?*
 - Uncomment lines 127-135 in the `national_park_test` file

Bonus: For any invalid inputs raise an **Exception** .

- First, **comment out** the following lines
 - `national_park_test.py`
 - lines 34-35
 - `visitor_test.py`
 - lines 23-24, 40-41, and 44-45
 - `trip_test.py`
 - lines 35-36, 49-50, 78-79, and 92-93
- Then, **uncomment** the following lines in the test files
 - `national_park_test.py`
 - lines 22-23, 26-27, and 38-39
 - `visitor_test.py`
 - lines 31-32, 48-49, 52-53
 - `trip_test.py`
 - lines 39-40, 53-54, 82-83, 96-97

